
Hardware Details and Setup for QuadraArm

1.1 Mechanical Design

The robotic arm we used was provided by our university lab. As shown below, it features four rotational joints and an optional gripper (which we didn't use in this project).

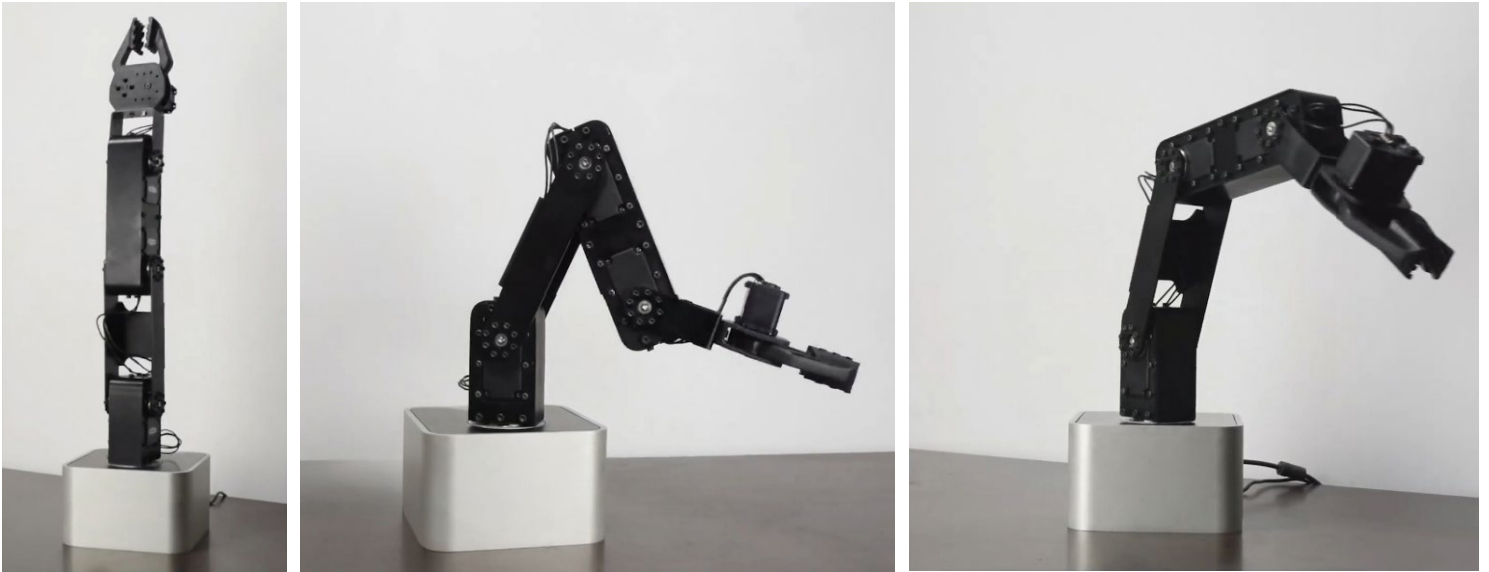


Figure 1: Actual image of the 4-DOF robotic arm

You can see each link and joint labeled in the image below.

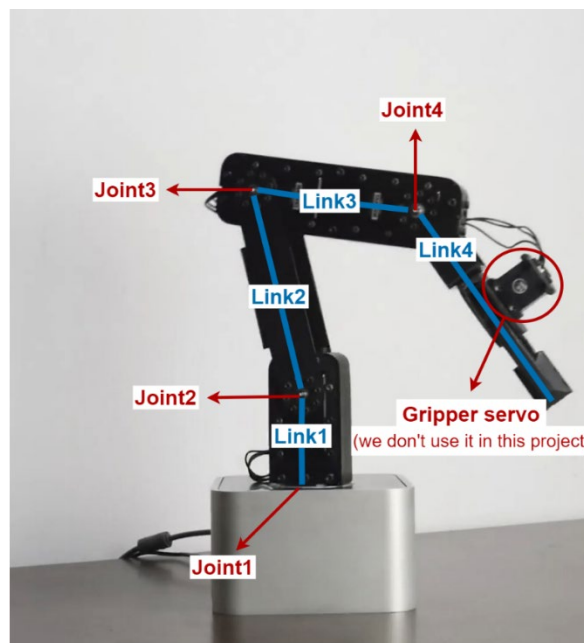


Figure 2: Arm with annotated links and joints

A simplified schematic is illustrated next, showing the rotational axes, link parameters, and coordinate system (X, Y, Z). These link lengths (denoted as l_1 to l_4) are later used in our kinematic equations. Note that l_4 includes the gripper length, although it's not actuated in our case.

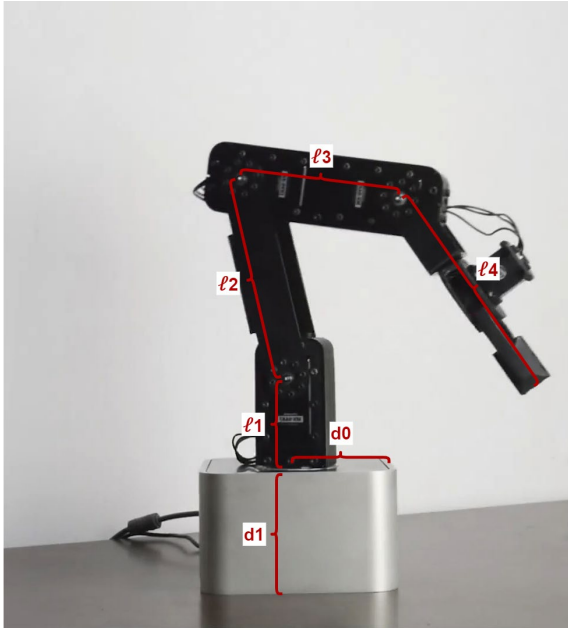
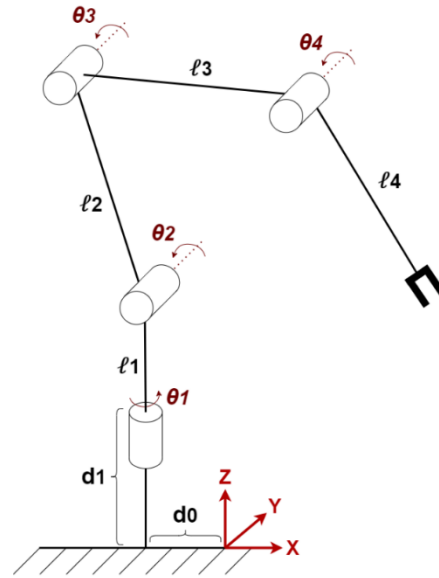


Figure 3: (a) Arm with annotated link lengths.



(b) Simplified representation of (a).

1.2 Actuators

The MX-28AT servo motors were used for the 1st, 3rd, and 4th joints, while the 2nd joint was driven by an MX-64AT servo (selected for its higher torque to handle the longer lever arm and payload when the robot is fully extended). Both models are manufactured by ROBOTIS and are specifically designed for robotic applications. Each motor includes a built-in microcontroller unit (MCU) and a configurable PID controller, enabling precise control over joint movements. The table below summarizes key technical specifications for both motor models.

Servo motor	MX-64AT	MX-28AT
Resolution	4096 pulse/rev	4096 pulse/rev
Stall torque (at 12V)	6 N.m (4.1A)	2.5 N.m (1.4A)
No load speed (at 12V)	63 RPM	55 RPM

Table 1: Specifications of servo motors

1.3 Electrical Schematic

Both the MX-28AT and MX-64AT servos feature a three-pin interface: VCC, GND, and DATA, with the DATA line operating over a half-duplex UART protocol. Thanks to the daisy-chaining feature of the MX series, the electronics setup is minimal, requiring that all three pins be chained from one motor to the next. Power is supplied via a 12V source (10A is sufficient), and communication is handled using a USB-to-TTL converter module, such as the official ROBOTIS U2D2 USB-to-serial adapter.

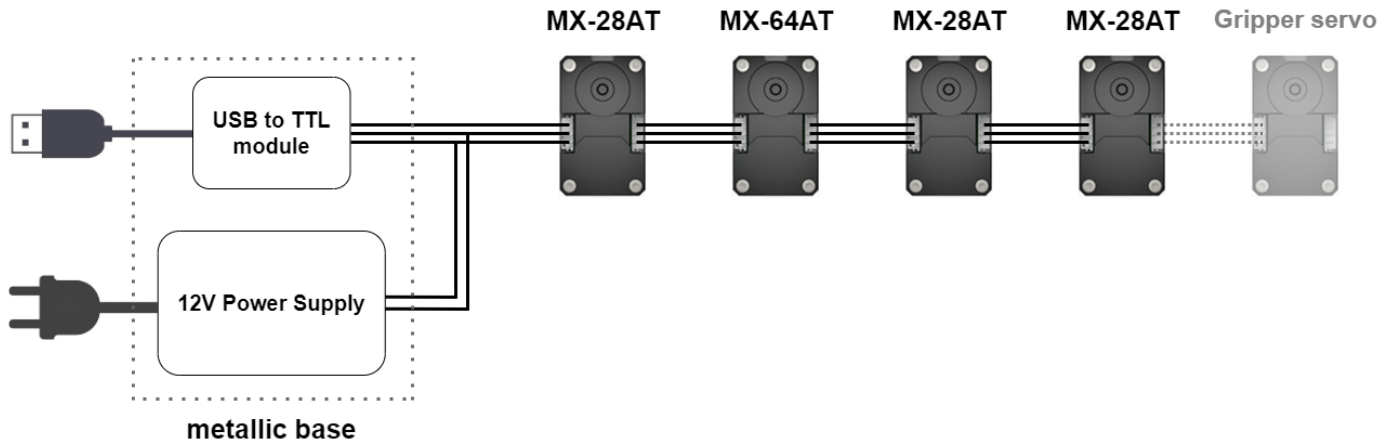


Figure 4: Wiring diagram of the robotic arm

1.4 PID Tuning

We start by exporting the robot's CAD model from SolidWorks and importing it into MATLAB using Simscape Multibody Link. This enables the construction of dynamic model in Simulink. Next, the joint actuators are modeled, and PID controllers are added in Simulink. The controller gains are tuned using either MATLAB's PID Tuner or optimization-based methods, depending on the performance objectives. After validation in simulation, the PID parameters are transferred to the physical robot (using the DYNAMIXEL Wizard tool, as introduced in the next section). Minor adjustments are made, if necessary, to account for real-world effects. In our case, the robot came with pre-defined PID gains, so tuning was not required.

2. Connecting to the Robot

Before interfacing with the robot, ensure proper hardware connection and driver installation. Once your USB-to-TTL module is recognized and a COM port appears in your system, it will serve as the communication bridge between your computer and the robot. All interaction is handled via Python code. The steps below will guide you through setup and initialization.

Step 1: Configure the DYNAMIXEL Servos

Install DYNAMIXEL Wizard 2.0 [[Tutorial Link](#)]. This graphical tool is essential for diagnosing and configuring DYNAMIXEL motors. Once installed, ensure that each motor responds correctly. Next, make sure the operating mode of all servos is set to 3 (Position Control Mode), and set the baudrate to 1000000 (recommended for smooth and stable communication). Finally, configure each motor with the following IDs:

- Motor at joint 1 → ID = 1
- Motor at joint 2 → ID = 2
- Motor at joint 3 → ID = 3
- Motor at joint 4 → ID = 4

Step 2: Install Python Dependencies

Install the DYNAMIXEL SDK v2.0 for Python [[Tutorial Link](#)]. This SDK provides a complete API for low-level motor control. You can find full documentation here [[Link](#)].

Also, install the required Python packages:

```
pip install numpy
pip install keyboard
```

Step 3: Clone the Code and Adjust Parameters

Clone the GitHub repository and open the `robotic_arm.py` file. Locate the `__init__` method and modify the following parameters based on your robot's configuration:

```
def __init__(self):
    self.baud_rate = 1000000
    self.max_position_limit = [ 90, 90, 90, 45] # theta1,theta2,theta3,theta4
    self.min_position_limit = [-90, -45, -70, -90]
    self.motor_velocity = 15 # 15 * 0.229 RPM (applied to all motors)
    self.d0 = 6 # cm
    self.d1 = 7.5 # cm
```

```
self.l1 = 6    # cm  
self.l2 = 13   # cm  
self.l3 = 11   # cm  
self.l4 = 14   # cm
```

These parameters define the mechanical dimensions and motion constraints of your robotic arm.

Step 4: Initialize the Robot

You are now ready to set the COM port and establish communication:

```
from robotic_arm import RobotArm4DOF  
from utility import *  
  
arm = RobotArm4DOF()  
arm.connect('COM3') # Replace 'COM3' with your actual COM port  
arm.init()
```

The `init()` method sets the motor position limits, applies the velocity profile, and enables torque for all motors.